Community Event Ticketing

A small, real-world web application that helps a community organizer publish a free event, register attendees, issue unique tickets, and check people in.



Student outcome
Move from idea to product requirements, functional requirements, technical requirements, and an MVP build plan.

Worked example Small scope Beginner friendly

A complete event flow, without platform bloat

The product serves a small community organization that currently manages RSVPs in spreadsheets and checks names manually at the door. The MVP replaces that fragile process with one simple, trackable flow.

Organizer creates event → **Guest registers** → **System issues ticket** → **Volunteer checks in**

Idea description

Neighbourhood Tickets is a lightweight web application for free workshops, fundraisers, cultural nights, and local meetups. An organizer publishes an event with capacity and registration details. Attendees reserve tickets and receive a unique ticket code. At the venue, a volunteer searches the attendee or enters the code to record check-in.

The first release supports one organization, one organizer account, free tickets only, and email confirmation. It is intentionally small enough for a student team to design and build.

Business value

- Reduces duplicate and missing registrations.
- Shows remaining capacity before an event fills.
- Speeds up the arrival desk.
- Produces a reliable attendance list.

Primary users

Organizer: publishes the event and monitors registrations.

Attendee: reserves a free ticket and receives confirmation.

Check-in volunteer: validates a ticket and marks attendance.

1
ORGANIZATION
No multi-tenant administration.

0
ONLINE PAYMENTS
Free registration keeps the MVP focused.

4
CORE SCREENS
Event page, form, confirmation, check-in.

Scope first: what the MVP will and will not do

A useful first release is defined as much by its exclusions as by its features. These boundaries protect the schedule and make the final demo credible.

In scope

- [ ] Organizer sign-in.
- [ ] Create, edit, publish, and cancel one-off events.
- [ ] Public event page with capacity status.
- [ ] Attendee registration for 1-4 tickets.
- [ ] Unique ticket code for each attendee order.
- [ ] Confirmation email with ticket details.
- [ ] Attendee list, search, and manual check-in.
- [ ] Registration and attendance counts.

Out of scope

- Paid tickets, refunds, taxes, or discount codes.
- Reserved seating or venue maps.
- Recurring events and season passes.
- Multiple organizations or organizer teams.
- Native iOS or Android applications.
- Marketing automation and social sharing.
- Advanced analytics or recommendation models.

> **Worked-example assumption:** The sample event is “Neighbourhood Skills Night,” a free, 80-seat evening workshop. Names, targets, and rules in this file are design inputs for learning, not real operational data.

Student build target

Area	Definition of small	Evidence in the demo
Users	Three roles; only organizer authentication is required.	Public guest flow and protected organizer/check-in pages.
Data	Four main entities: event, order, attendee, check-in.	Seeded event and persistent registrations.
Integrations	One email provider; no payment gateway.	Confirmation email or development email preview.
Reporting	Counts and exportable attendee list.	Registered, capacity remaining, and checked-in totals.

Learning goals

Students practice scope control, requirements traceability, relational data modelling, form validation, role-based access, transactional writes, and a deployment-ready definition of done.

Prompt 1: create the Product Requirements Document

Copy this prompt into your preferred model. Replace the double-braced variables only when you want a different event type, organization, or capacity.

Role: Act as a senior product manager helping a beginner development team.

Task: Write a concise Product Requirements Document (PRD) for a small web application called “Neighbourhood Tickets.” It serves {{ORGANIZATION_TYPE}} and manages free community events such as {{EVENT_EXAMPLES}}.

Business problem: Registrations are currently collected through messages and spreadsheets. The organizer cannot reliably enforce capacity, send consistent confirmations, or check attendees in quickly.

MVP flow: An organizer creates and publishes a one-off event. A guest opens the public event page, reserves 1-4 free tickets, and receives a unique ticket code by email. A volunteer searches by name/email or enters the code and marks the order as checked in.

Constraints:
- One organization and one organizer account.
- Free tickets only; no payments or refunds.
- No reserved seating, recurring events, native mobile app, or advanced analytics.
- Default event capacity: {{CAPACITY}}, but the organizer can change it.
- The product must be usable on phones and desktop browsers.

Required sections:
1. Executive summary
2. Problem statement and opportunity
3. Users and jobs to be done
4. Goals and measurable success indicators
5. Assumptions and constraints
6. In-scope and out-of-scope features
7. End-to-end user journeys
8. Business rules
9. Risks and mitigations
10. MVP release criteria

Use plain language. Make every success indicator testable. Do not add features outside the stated constraints. Mark any unresolved decision as an open question rather than inventing an answer.

Suggested variables

{{ORGANIZATION_TYPE}} = neighbourhood association
{{EVENT_EXAMPLES}} = workshops, cultural nights, and local meetups
{{CAPACITY}} = 80

Quality check

- ☐ Goals describe outcomes, not features.
- ☐ Every feature fits the MVP flow.
- ☐ Open questions are visible.
- ☐ Release criteria can be demonstrated.

Prompt 2: write functional requirements

Use the approved PRD as the source of truth. The model should convert product intent into numbered, testable behaviour without designing the implementation.

Role: Act as a business analyst writing functional requirements for a small web application.

Input: Use the approved PRD for “Neighbourhood Tickets” pasted below.

{{PASTE_APPROVED_PRD}}

Task: Produce functional requirements for the MVP. Cover these feature areas: organizer access, event management, public event display, attendee registration, capacity control, ticket generation, email confirmation, attendee search, check-in, cancellation, and summary counts.

For every requirement include:
- A unique ID in the form FR-01, FR-02, and so on.
- Requirement statement using “The system shall...”
- Primary actor.
- Preconditions, trigger, main flow, and alternate/error flows.
- Data captured or displayed.
- Business rules and validation.
- Acceptance criteria written as Given / When / Then.

Rules:
- Keep the solution to one organization and free tickets.
- Do not specify programming languages, frameworks, databases, or vendors.
- Include privacy-conscious behaviour: collect only name and email, and do not expose the attendee list publicly.
- Prevent registrations above capacity and repeat check-in.
- Preserve a cancelled event and its registration history for the organizer.

End with a traceability table mapping each functional requirement to the PRD goal and user journey it supports. Identify gaps or conflicts as open questions; do not resolve them silently.

Why this prompt works: It separates *what the system must do* from *how developers will build it*, while requiring acceptance criteria that can become test cases.

Review before accepting the output

- ☐ IDs are unique and sequential.
- ☐ Every flow includes failure states.
- ☐ Capacity is enforced atomically.
- ☐ Private data stays private.
- ☐ Acceptance criteria are observable.
- ☐ No technical stack appears.

Prompt 3: write technical requirements

The technical prompt should start only after the PRD and functional requirements are approved. It asks for a minimal architecture that can satisfy the documented behaviour.

Role: Act as a senior full-stack engineer designing a beginner-friendly MVP.

Inputs: Use the approved PRD and functional requirements for “Neighbourhood Tickets.”

{{PASTE_APPROVED_PRD}}

{{PASTE_APPROVED_FUNCTIONAL_REQUIREMENTS}}

Task: Write technical requirements for a small, deployable web application. Prefer the simplest architecture that meets the requirements.

Required sections:
1. Architecture overview and component responsibilities
2. Recommended stack with brief rationale
3. Data model, fields, types, relationships, constraints, and indexes
4. API or server-action contracts, including request, response, authorization, validation, and error cases
5. Authentication and role-based access
6. Ticket-code generation and check-in concurrency rules
7. Email-delivery approach with a development fallback
8. Security, privacy, accessibility, logging, backup, and performance requirements
9. Environment configuration and deployment
10. Unit, integration, and end-to-end test strategy
11. Requirement-to-component traceability

Technical guardrails:
- Use a single responsive web application and one relational database.
- Keep attendee pages public; protect organizer and check-in functions.
- Enforce capacity and check-in integrity in database transactions.
- Never store passwords directly.
- Do not include payment, microservices, message queues, or native apps.
- Describe vendor-neutral requirements first; place specific tools in a recommended implementation subsection.

For each non-functional target, include a measurable threshold or a clear verification method. Call out decisions that require team approval.

Keep it teachable

Reject unnecessary infrastructure. A monolith plus one database is the correct trade-off for this project.

Keep it traceable

Every endpoint, table, and test should support a numbered functional requirement.

PRD: a reliable registration desk for small events

Executive summary

Neighbourhood Tickets gives a community organizer one dependable place to publish a free event, control capacity, confirm reservations, and record attendance. The MVP replaces informal messages and spreadsheet lists with a public registration flow and a protected organizer workspace.

Problem and opportunity

Manual registration creates duplicate entries, inconsistent confirmations, accidental overbooking, and slow check-in. Organizers need a low-training tool that works for occasional events and produces a trustworthy attendance record.

Users and jobs to be done

Organizer: “When I plan an event, I need one live attendee list so I can manage capacity and prepare the venue.”

Attendee: “When I decide to attend, I need a quick reservation and clear confirmation so I know my place is saved.”

Volunteer: “When guests arrive, I need to find and check them in quickly so the entrance keeps moving.”

Product goals

1. Provide a complete registration in under two minutes for a typical attendee.
2. Prevent confirmed ticket quantity from exceeding event capacity.
3. Give the organizer a current count of registrations and check-ins.
4. Allow a volunteer to locate a valid registration by code, name, or email.

Success indicators

Indicator	MVP target
Registration completion	At least 80% in usability test
Median test completion time	Under 2 minutes
Capacity breaches	Zero in concurrency test
Successful test check-ins	At least 95%

> **Measurement note:** These are MVP acceptance targets for classroom testing. They are not claims about real users or production performance.

PRD: journeys, business rules, and release boundary

Journey A · Publish

1. Organizer signs in.
2. Creates an event with title, summary, date/time, venue, capacity, and registration deadline.
3. Reviews and publishes.
4. Copies the public event link.

Journey B · Register

1. Guest opens the event page.
2. Chooses 1-4 tickets and submits name and email.
3. System confirms capacity.
4. Guest sees and receives a ticket code.

Journey C · Check in

1. Volunteer opens the protected check-in view.
2. Searches by ticket code, name, or email.
3. Confirms the matching order.
4. System records check-in and blocks repeats.

Journey D · Cancel

1. Organizer cancels a published event.
2. Public page shows cancellation.
3. New registration closes.
4. History remains available to organizer.

Business rules

ID	Rule
BR-01	A published event must have a future start time, venue, capacity of at least 1, and registration deadline no later than its start.
BR-02	Each order reserves 1-4 tickets and requires an attendee name and valid email address.
BR-03	Total confirmed ticket quantity cannot exceed capacity.
BR-04	Each confirmed order has one unique, non-sequential ticket code.
BR-05	An order can be checked in once. A repeated attempt shows the original check-in time.
BR-06	Cancelling an event closes registration but does not delete registrations or check-in history.

Key risks

- **Overbooking:** use transactional capacity checks.
- **Email failure:** always show confirmation on screen and allow organizer lookup.
- **Privacy:** restrict attendee data to authorized pages.

MVP release criteria

- ☐ All P0 requirements pass.
- ☐ Capacity race test passes.
- ☐ Organizer can export attendance.
- ☐ Keyboard-only critical flow passes.
- ☐ Production backup and restore are tested.

Functional requirements: organizer and event flow

Each requirement states observable behaviour. P0 items are required for release; P1 is useful but may move to a later iteration.

FR-01
P0

The system shall authenticate the organizer before showing event, attendee, export, or check-in functions.

Acceptance: Given a signed-out visitor, when a protected URL is opened, then the visitor is redirected to sign-in and no private attendee data is returned.

FR-02
P0

The system shall let the organizer create and edit a draft event with title, summary, start/end time, venue, capacity, registration deadline, and contact email.

Acceptance: Given valid values, when the organizer saves, then the draft persists and can be reopened. Invalid or missing values are identified beside the affected field.

FR-03
P0

The system shall publish only a valid future event. Publishing creates a stable public URL and changes status from Draft to Published.

Acceptance: Given a valid draft, when Publish is confirmed, then the public page is available. Given an invalid draft, publication is blocked with corrective guidance.

FR-04
P0

The system shall display a public event page containing event details, registration status, tickets remaining, and an accessible registration form.

Acceptance: Given a published open event, when a guest opens its URL, then accurate details and the form appear without sign-in.

FR-05
P0

The system shall allow the organizer to cancel an event after an explicit confirmation step.

Acceptance: Given a published event, when cancellation is confirmed, then status becomes Cancelled, registration is disabled, the public page shows the cancellation, and prior records remain accessible.

FR-06
P1

The system shall let the organizer export the attendee list as a comma-separated values file containing name, email, quantity, status, ticket code, and check-in time.

Acceptance: Given registrations exist, when Export is selected, then a correctly headed file downloads with one row per order.

Functional requirements: registration and arrival

FR-07
P0

The system shall register an attendee for 1-4 tickets using name and email, after validating the event is open and enough capacity remains.

Acceptance: Given three seats remain, when a guest requests four, then no order is created and the page asks for a smaller quantity.

FR-08
P0

The system shall reserve capacity atomically when a registration is confirmed.

Acceptance: Given one seat remains and two guests submit simultaneously, when both requests complete, then only one ticket is confirmed and capacity is never exceeded.

FR-09
P0

The system shall issue a unique ticket code and show a confirmation page with event details, ticket quantity, and the code.

Acceptance: Given registration succeeds, when confirmation loads, then the code is present, unique across orders, and not derived from a sequential database ID.

FR-10
P0

The system shall send a confirmation email to the registered address and record delivery state without delaying the on-screen result.

Acceptance: Given email delivery fails, when registration succeeds, then the ticket remains valid, the organizer can find it, and the failure is logged for follow-up.

FR-11
P0

The system shall provide protected attendee search by exact ticket code or partial name/email, scoped to the selected event.

Acceptance: Given matching records, when a volunteer searches, then only the selected event's results appear with registration and check-in status.

FR-12
P0

The system shall check in a valid order once and record time plus acting organizer account.

Acceptance: Given an unchecked valid order, when Check in is confirmed, then its state changes immediately. A repeat attempt is blocked and shows the original time.

FR-13
P0

The system shall display event totals for capacity, confirmed tickets, remaining tickets, checked-in orders, and checked-in guests.

Acceptance: Given registrations and check-ins change, when the organizer refreshes, then totals reconcile with stored records.

Technical requirements: one application, one database

A modular monolith is the smallest architecture that supports public registration, protected administration, transactional capacity control, and deployment.

Responsive browser UI
Public event, registration, confirmation, admin, check-in

→

Web application
Routes, authorization, validation, business logic, email adapter

→

Relational database
Users, events, orders, check-ins, audit fields

Recommended implementation

Layer	Choice
Application	TypeScript web framework with server-rendered pages and server actions or REST routes
Database	PostgreSQL with migrations
Authentication	Managed email sign-in or framework session library
Email	Transactional email adapter; file/log preview in development
Hosting	Single web service plus managed PostgreSQL

Component responsibilities

- **Event module:** draft validation, publication, status, public retrieval.
- **Registration module:** input validation, capacity transaction, ticket-code creation.
- **Notification module:** confirmation rendering, send attempt, delivery status.
- **Check-in module:** protected search and idempotent attendance recording.
- **Reporting module:** totals and CSV export.

Decision to approve

Choose one framework and one authentication approach before coding. Preserve the vendor-neutral contracts on the next page.

Technical quality targets

Area	Requirement	Verification
Performance	Key pages respond within 2 seconds at the 95th percentile under 25 concurrent test users.	Automated load test in staging.
Accessibility	Critical flows meet Web Content Accessibility Guidelines 2.2 Level AA.	Automated scan plus keyboard review.
Reliability	Registration transaction never confirms above capacity.	Concurrent integration test.
Recovery	Database backup can be restored to staging.	Documented restore drill before launch.

Technical requirements: data and service contracts

Relational data model

Entity	Required fields	Constraints and indexes
User	id, email, display_name, role, created_at	Unique normalized email; role limited to organizer.
Event	id, slug, title, summary, starts_at, ends_at, venue, capacity, registration_deadline, contact_email, status, timestamps	Unique slug; capacity > 0; indexed status/start time.
TicketOrder	id, event_id, attendee_name, attendee_email, quantity, ticket_code_hash, status, email_status, created_at	Quantity 1-4; unique ticket-code hash; indexed event/email.
CheckIn	id, order_id, checked_in_at, checked_in_by	Unique order_id; foreign keys to order and user.

Service contracts

```
GET /events/:slug
200 public event summary
404 unknown or unavailable

POST /events/:slug/registrations
body: name, email, quantity
201 confirmation + ticket code
409 capacity unavailable
422 validation errors

GET /admin/events/:id/attendees?q=
authorized search results

POST /admin/orders/:id/check-in
200 check-in record
409 already checked in
```

Transaction rules

1. Begin a database transaction.
2. Lock or atomically update the event capacity calculation.
3. Recheck event status, deadline, and remaining seats.
4. Create the order and ticket-code hash.
5. Commit, then request email delivery.

Ticket-code handling

Generate at least 128 bits of cryptographically secure randomness. Show the encoded code to the attendee, but store a one-way hash. The code must not reveal an order ID or attendee information.

Error shape

```
{
  "error": {
    "code": "CAPACITY_UNAVAILABLE",
    "message": "Only 3 tickets remain."
  }
}
```

Technical requirements: security, privacy, and testing

Security and privacy

- Require HTTPS in production and secure, HTTP-only, same-site session cookies.
- Authorize every organizer route on the server; hiding a button is not authorization.
- Validate and normalize input on the server; encode output by context.
- Apply request throttling to registration, sign-in, and ticket-code lookup.
- Protect state-changing requests from cross-site request forgery.
- Do not write attendee name, email, or raw ticket code to application logs.
- Keep attendee data only as long as the organization requires; document the retention period before launch.

Accessibility

- Use semantic headings, labels, landmarks, and native controls.
- Provide visible keyboard focus and logical tab order.
- Associate validation errors with fields and announce the error summary.
- Do not communicate event status by colour alone.

Test strategy

Level	Minimum coverage
Unit	Validation, status rules, remaining-capacity calculation, ticket-code encoding.
Integration	Publish, register, full-capacity conflict, cancellation, check-in idempotency, authorization.
End to end	Create/publish; public register; organizer search/check-in; export.
Non-functional	Keyboard path, automated accessibility scan, 25-user load test, backup restore.

Operations

- Separate development, staging, and production configuration.
- Store secrets in the hosting platform's secret manager.
- Run schema migrations as a controlled deployment step.
- Log request ID, route, status, duration, and error class without personal data.
- Expose a health endpoint that checks application and database connectivity.

> **Open decision:** The organization must approve the attendee-data retention period and the sender identity for confirmation email before production launch.

From requirements to a working demo in four milestones

1

Foundation

Create the repository, application shell, database schema, migrations, seed event, organizer authentication, and protected routes.

Demo: organizer signs in and opens an empty event dashboard.

2

Publish

Build event form validation, draft save, publication rules, public event route, and cancellation state.

Demo: organizer publishes and shares a valid event page.

3

Register

Add capacity-safe registration, confirmation page, secure ticket code, email adapter, and summary totals.

Demo: guest reserves; full-event attempt is blocked.

4

Arrive and release

Add search, idempotent check-in, CSV export, accessibility fixes, tests, deployment, and restore drill.

Demo: volunteer checks in a guest and organizer exports attendance.

Traceability snapshot

Product goal	Functional requirements	Technical evidence
Fast registration	FR-04, FR-07, FR-09, FR-10	Responsive public routes; server validation; email adapter; end-to-end timing test
No overbooking	FR-07, FR-08, FR-13	Database transaction; capacity constraint logic; concurrent integration test
Accurate attendance	FR-11, FR-12, FR-13	Scoped search; unique check-in constraint; totals query
Protected attendee data	FR-01, FR-06, FR-11	Server authorization; safe export; redacted logs; access tests

Student definition of done

- ☐ Requirements are implemented or explicitly deferred.
- ☐ All P0 automated tests pass.
- ☐ A fresh deployment completes the critical flow.
- ☐ The README explains setup and environment variables.

> **Build principle:** Finish the smallest complete journey first. A narrow product that works end to end teaches more than a wide product made of unfinished screens.